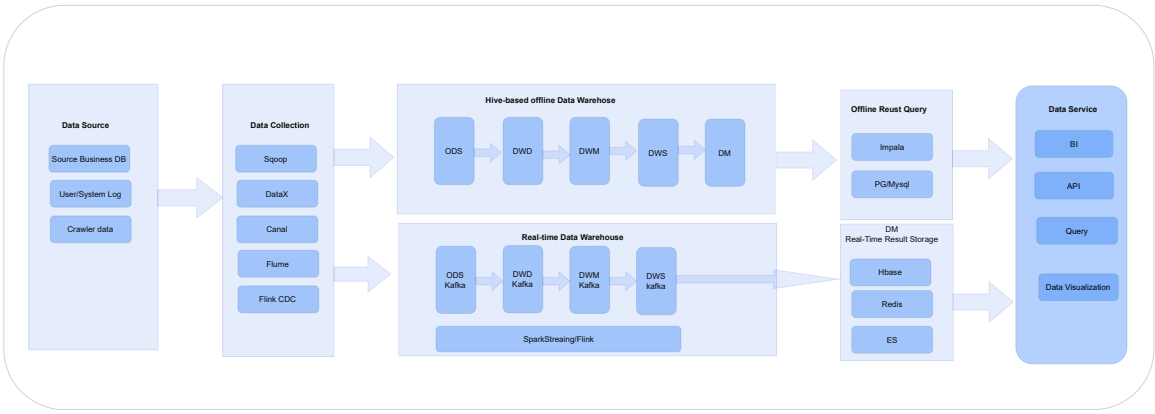
Data Platform

Architecture evolution

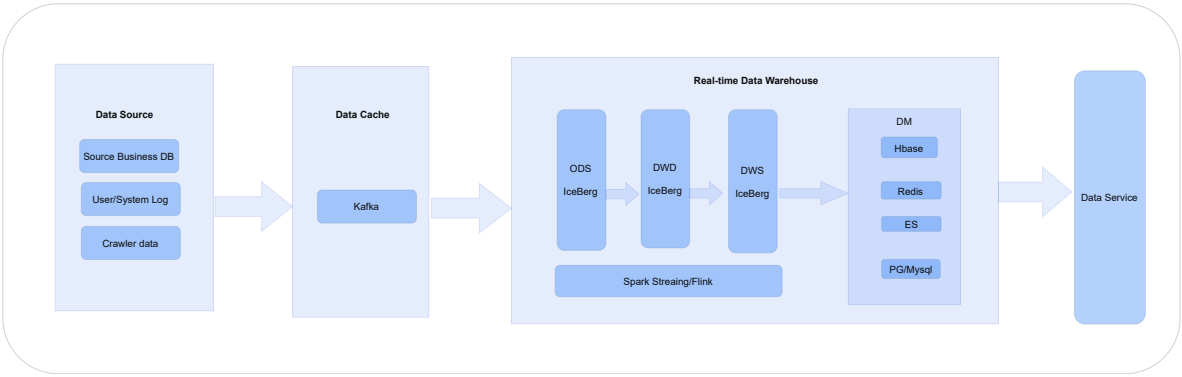
Lambda



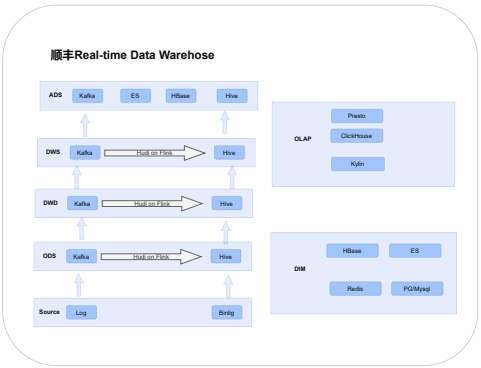
Kappa

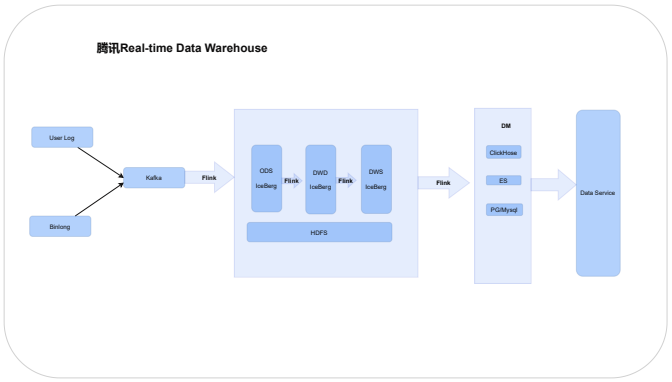


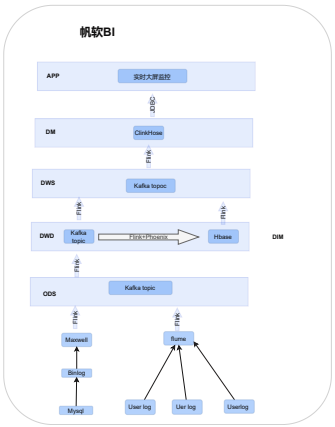
Data Lakehouse

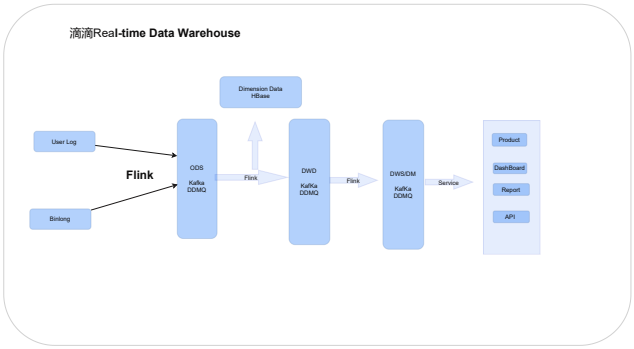

Data Source
Source Business DB
User/System Log
Crawler data
Data Cache
Kafka
Real-time Data Warehouse
ODS
IceBerg
DWD
IceBerg
DWS
IceBerg
DM
Hbase
Redis
ES
PG/Mysql
Spark Streaing/Flink
Data Service

eg


顺丰Real-time Data Warehose
ADS
Kafka
ES
HBase
Hive
DWS
Kafka
Hudi on Flink
Hive
DWD
Kafka
Hudi on Flink
Hive
ODS
Kafka
Hudi on Flink
Hive
Source
Log
Binlg
OLAP
Presto
ClickHouse
Kylin
DIM
HBase
ES
Redis
PG/Mysql


腾讯Real-time Data Warehouse
User Log
Binlong
Kafka
Flink
ODS
IceBerg
Flink
DWD
IceBerg
Flink
DWS
IceBerg
Flink
HDFS
DM
ClickHose
ES
PG/Mysql
Data Service


帆软BI
APP
实时大屏监控
JDBC
DM
ClinkHose
Flink
DWS
Kafka topoc
Flink
Flink
DWD
Kafka topic
Flink+Phoenix
Hbase
DIM
Flink
ODS
Kafka topic
Flink
Flink
Maxwell
Binlog
Mysql
flume
User log
Uer log
Userlog


滴滴Real-time Data Warehouse
Dimension Data
HBase
User Log
Binlong
Flink
ODS
Kafka
DDMQ
Flink
DWD
Kafka
DDMQ
Flink
DWS/DM
Kafka
DDMQ
Service
Product
DashBoard
Report
API

Architecture	Disadvantages	湖仓一体对其缺点的解决
Lambda	1. 需开发两套代码实现同样需求 2. 集群资源占用增多 3. 离线与实时结果不一致 4. 批量计算可能存在T + 1计算不完的情况 5. 服务器存储需求大	未直接针对Lambda架构这些缺点专门解决，但通过存储统一、支持OLAP等， 在一定程度上简化架构和提升数据处理能力，间接减少对资源和存储的压力
Kappa	1. Kafka无法支撑海量数据存储 2. Kafka无法支持高效的OLAP 3. 无法复用数据血缘管理体系 4. Kafka不支持update/upsert	1. 存储统一解决Kafka存储量小问题 2. 任意分层都可OLAP数据分析，解决OLAP支持问题 3. 复用同一套血缘关系，解决血缘管理体系复用问题 4. 实时数据更新，解决Kafka不支持update/upsert问题

Data Lake Storage-layer format

Table Format

1. Delta Lake
2. Apache Hudi
3. Apache Iceberg

Column - based Storage Formats

- Parquet
- ORC

Binary Serialization Formats

- Avro

Text Formats

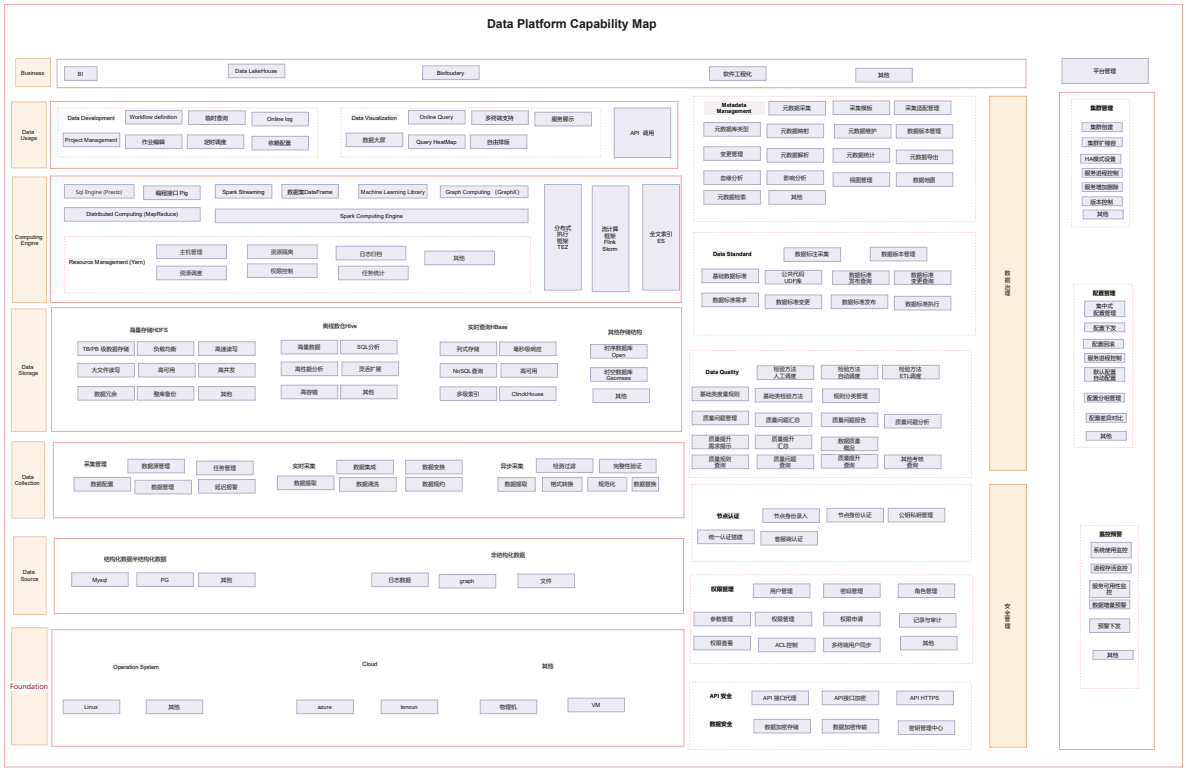
- JSON

- CSV

Database & Data Warehose

Function	Database (OLTP)	Data Warehouse (OLAP)
Data Scope	Current - state data	Store complete data that reflects historical changes
Data Change	Support frequent insert, delete, update, and query operations	Can insert, query, no update or delete operations
Application Scenarios	Oriented to business transaction processes	Oriented to analysis, focusing on decision - making analysis
Data Processing Volume	Frequent, small batches, high concurrency, low latency	Infrequent, large batches, high throughput, with latency
Design Theory	Follow the three normal forms of the database, avoid redundancy	Violate normal forms, appropriate redundancy
Modeling Method	ER entity - relationship modeling (normal form modeling)	Normal form modeling + dimensional modeling

H5



TenXun Cloud

Azure Cloud